Rambus

Security IP
EIP-130

Root of Trust engine
Driver Library v3.1

User Guide

Document Revision: A
Document Date: 2022-02-09
Document Number: 007-130310-307
Document Status: Accepted

© Rambus Inc. • rambus.com

Copyright 2015-2022 Rambus Inc. This document contains information which is proprietary and confidential, and which is protected under patents, copyrights, and/or other IP rights of Rambus Inc. If you are not the intended recipient of this material, please destroy this document and inform Rambus at +1 408 463 8000 or sipsupport@rambus.com immediately.

Rambus Inc. Corporate Headquarters
4453 North First Street, Suite 100
San Jose, CA 95134
Phone: +1 408-462-8000
Website : https://www.rambus.com/
Contact : sipsupport@rambus.com

Rambus ROTW Holding B.V.
Boxtelseweg 26A
5261 NE Vught
The Netherlands
Phone: +31-73-6581900

Table of Contents

Table of Contents 1
List of Tables 4
List of Figures 4
Document Revision History 5
1 Introduction 6
1.1 Purpose 6
1.2 Scope 6
1.3 Related Documents 6
1.4 Target Audience 6
1.5 Abbreviations, Acronyms and Definitions 6
2 Functional Overview 8
2.1 Architecture Overview 8
2.2 EIP-130 HW Structure and Terminology 9
2.3 Driver Framework 9
2.4 Driver Library Configuration 9
2.5 Endianness 9
2.5.1 Endianness Conversion by the Host Processor 10
2.6 Concurrent Context Synchronization: Registers and Mailboxes 10
2.6.1 API Use Order 10
2.7 Concurrent Context Synchronization: Token Helper functions 11
2.8 Deployment 11
3 Provided Services 12
3.1 Register API Functions Summary 12
3.2 Mailbox API Functions Summary 12
3.3 Firmware API Functions Summary 13
3.4 Token Helper API Summary 14
4 Configuration Parameters 22
4.1 General settings 22
4.1.1 EIP130_STRICT_ARGS 22
4.1.2 EIP130_HW_V2 22
4.1.3 EIP130_HW_V3 22
4.1.4 EIP130_ENABLE_FIRMWARE_LOAD 22

4.1.5 EIP130_ALLOW_MASTER_NONSECURE 22
4.2 Token Helper functions 22
4.2.1 NOP: EIP130_ENABLE_NOP 22
4.2.2 Encryption: EIP130_ENABLE_SYM_ALGO_AES 22
4.2.3 Encryption: EIP130_ENABLE_SYM_ALGO_AES_XTS 22
4.2.4 Encryption: EIP130_ENABLE_SYM_ALGO_AES_CCM 22
4.2.5 Encryption: EIP130_ENABLE_SYM_ALGO_AES_GCM 23
4.2.6 Encryption: EIP130_ENABLE_SYM_ALGO_AES_F8 23
4.2.7 Encryption: EIP130_ENABLE_SYM_ALGO_DES 23
4.2.8 Encryption: EIP130_ENABLE_SYM_ALGO_3DES 23
4.2.9 Encryption: EIP130_ENABLE_SYM_ALGO_CHACHA20 23
4.2.10 Encryption: EIP130_ENABLE_SYM_ALGO_ARIA 23
4.2.11 Encryption: EIP130_ENABLE_SYM_ALGO_ARIA_CCM 23
4.2.12 Encryption: EIP130_ENABLE_SYM_ALGO_ARIA_GCM 23
4.2.13 Encryption: EIP130_ENABLE_SYM_ALGO_SM4 23
4.2.14 Hash/MAC: EIP130_ENABLE_HASHMAC_LAYOUT_V2 23
4.2.15 Hash: EIP130_ENABLE_SYM_ALGO_SHA512 23
4.2.16 Hash: EIP130_ENABLE_SYM_ALGO_SHA3 23
4.2.17 Hash: EIP130_ENABLE_SYM_ALGO_SM3 23
4.2.18 MAC: EIP130_ENABLE_SYM_ALGO_POLY1305 24
4.2.19 TRNG: EIP130_ENABLE_TRNG_CUTOFF 24
4.2.20 TRNG: EIP130_ENABLE_DRBG_FORCESEED 24
4.2.21 TRNG: EIP130_ENABLE_SYM_ALGO_ARIA_DRBG 24
4.2.22 Special Functions: EIP130_ENABLE_SF_MILENAGE 24
4.2.23 Symmetric Key Wrap: EIP130_ENABLE_SYM_KEYWRAP 24
4.2.24 Symmetric Key Wrap: EIP130_ENABLE_ENCRYPTED_VECTOR 24
4.2.25 Asset Management: EIP130_FW_ASSET_LIFETIME_MANAGEMENT 24
4.2.26 Asset Management: EIP130_FW_ASSETPOLICY_V2 24
4.2.27 Asset Management: EIP130_ENABLE_CPIF 24
4.2.28 Asset Management: EIP130_ENABLE_ASSET_STORE_RESET 24
4.2.29 Public Key: EIP130_ENABLE_EXPLICITDIGEST 24
4.2.30 Public Key: EIP130_ENABLE_ASYM_ALGO_DH 25
4.2.31 Public Key: EIP130_ENABLE_SYM_ALGO_ECDH 25
4.2.32 Public Key: EIP130_ENABLE_ASYM_ALGO_CURVE25519 25
4.2.33 Public Key: EIP130_ENABLE_ASYM_ALGO_ECDSA 25
4.2.34 Public Key: EIP130_ENABLE_ASYM_ALGO_EDDSA 25
4.2.35 Public Key: EIP130_ENABLE_ASYM_ALGO_ECC_ELGAMAL 25
4.2.36 Public Key: EIP130_ENABLE_ASYM_ALGO_RSA 25
4.2.37 Public Key: EIP130_ENABLE_ASYM_ALGO_ECIES 25
4.2.38 Public Key: EIP130_ENABLE_SYM_ALGO_SM2 25

4.2.39 eMMC: EIP130_ENABLE_EMMC 25
4.2.40 External Service: EIP130_ENABLE_EXTSERVICE 25
4.2.41 System: EIP130_ENABLE_FIRMWARE_SLEEP 25
4.2.42 System: EIP130_ENABLE_FIRMWARE_HIBERNATION 26
4.2.43 System: EIP130_ENABLE_SYSTEM_ENTERFIPSMODE 26
4.3 Register, Mailbox and Firmware API functions 26
4.3.1 EIP130_REMOVE_MAILBOXACCESSVERIFY 26
4.3.2 EIP130_REMOVE_MAILBOXGETOPTIONS 26
4.3.3 EIP130_REMOVE_MAILBOXACCESSCONTROL 26
4.3.4 EIP130_REMOVE_MAILBOXLINK 26
4.3.5 EIP130_REMOVE_MAILBOXLINKRESET 26
4.3.6 EIP130_REMOVE_MAILBOXUNLINK 26
4.3.7 EIP130_REMOVE_MAILBOXCANWRITETOKEN 26
4.3.8 EIP130_REMOVE_MAILBOXRAWSTATUS 26
4.3.9 EIP130_REMOVE_MAILBOXRESET 26
4.3.10 EIP130_REMOVE_MAILBOXLINKID 26
4.3.11 EIP130_REMOVE_MAILBOXOUTID 27
4.3.12 EIP130_REMOVE_FIRMWARECHECK 27
4.3.13 EIP130_REMOVE_FIRMWAREDOWNLOAD 27
4.3.14 EIP130_REMOVE_MODULEFIRMWAREWRITTEN 27
4.3.15 EIP130_REMOVE_MODULEGETOPTIONS 27
4.3.16 EIP130_REMOVE_MODULEGETSTATUS 27
4.3.17 EIP130_REMOVE_VERSION_OPTION 27

List of Tables

Table 1 *EIP-130 Driver Library* Mailbox API State Machine 10
Table 2 *EIP-130 Driver Library* Register API 12
Table 3 *EIP-130 Driver Library* Mailbox API 13
Table 4 *EIP-130 Driver Library* Firmware API 13
Table 5 *EIP-130 Driver Library* Token Helper API 14

List of Figures

Figure 1 Driver Library Integration Overview 8
Figure 2 *EIP-130* interface 9
Figure 3 *EIP-130 Driver Library* Mailbox API State Machine 11

Document Revision History

Doc Rev	Page(s) Section(s)	Date (Y-M-D)	Author	Purpose of Revision
A	All	2022-02-09	MHO	• First release for v3.1, based on v3.0 RevC.

1 Introduction

1.1 Purpose

The purpose of this document is to guide software developers in using the *EIP-130 Driver Library* to access the services of the *EIP-130* hardware also referred to as *RT-1xx*, or *VaultIP*. This document explains in detail how to configure and use the EIP-130 Driver Library for an optimal use of the *EIP-130* hardware.

Throughout this document, the *EIP-130* hardware module is also referred to as the *EIP-130*. The terms 'Host' and 'CPU' are used interchangeably to mean the CPU on which the driver software including the Driver Library runs.

1.2 Scope

The scope of this document is the usage details of the *EIP-130 Driver Library*. This library implements the control functionality for the data I/O operations for the *EIP-130*.

This document is structured as follows.

Chapter 2 contains a functional overview of the *EIP-130 Driver Library*.

Chapter 3 describes the Driver Library API in detail.

Chapter 4 describes the Driver Library configuration parameters in detail.

The reader is also advised to refer to the API header files [1] listed in section 1.3 for details on parameters for each of the functions.

Refer to the package Release Notes for the supported *EIP-130* hardware/firmware versions.

1.3 Related Documents

Ref	Document	Doc. Number
[1]	EIP-130 Driver Library source code, Token Helper files (`eip130_token_*.h` and `eip130_asset_policy.h`) and API header file (`eip130.h` and `eip130_level0.h`)	N.A.
[2]	Security IP Driver Framework Porting Guide	007-900450-304

This information is correct at the time of document release. Rambus reserves the right to update the related documents without updating this document. Please contact Rambus for the latest document revisions.

The preferred method for getting technical support is to use our online support system at https://sipsupport.rambus.com. If you do not have an account yet for this system, please contact Rambus technical support (sipsupport@rambus.com).

1.4 Target Audience

This document is intended for driver developers. This document will assist the developer in writing a device driver for the *EIP-130.*

1.5 Abbreviations, Acronyms and Definitions

Acronym	Description
3DES	Triple-DES
AES	Advanced Encryption Standard
AMP	Asymmetric Multi-Processing
API	Application Program Interface
ARIA	South-Koran cipher algorithm
CBC	Cipher Block Chaining

CCM	Counter with CBC-MAC
ChaCha20	Stream cipher created by Daniel J. Bernstein
CMAC	Cipher MAC
CTR	CounTeR
DDK	Driver Development Kit
DES	Data Encryption Standard
DMA	Direct memory Access
DRBG	Deterministic Random Bit Generator
ECB	Electronic CodeBook
ECC	Elliptic Curve Cryptography
ECDSA	Elliptic Curve Digital Signature Algorithm
EdDSA	Edwards-curve Digital Signature Algorithm
EIP	Embedded Intellectual Property
FW	Firmware
GCM	Galois/Counter Mode
HMAC	Hash MAC
HUK	Hardware Unique Key (Root Key)
HW	Hardware
IC	Interrupt Coalescing
ICM	Integer Counter Mode
KAT	Known-Answer Test
MAC	Message Authentication Code
Milenage	3GPP authentication and key generation functions
NOP	No OPeration
NRBG	Non-deterministic Random Bit Generator
OS	Operating System
OTP	One-Time Programmable
PKA	Public Key Accelerator
PKCS	Public-Key Cryptography Standards
PKI	Public Key Infrastructure
Poly1305	Cryptographic message authentication code created by Daniel J. Bernstein
PSS	Probabilistic Signature Scheme
RSA	Rivest, Shamir, Adleman
TRNG	True Random Number Generator
SIV	Synthetic Initialization Vector
SM2	Chinese asymmetric algorithm
SM3	Chinese symmetric hash algorithm
SM4	Chinese symmetric cipher algorithm
SMP	Symmetric Multi-Processing
SW	Software
UP	UniProcessor
VAL	VaultIP Abstraction Layer (API)
VEX	VaultIP EXchange
XEX	XOR Encrypt XOR
XOR	eXclusive OR
XTS	XEX Tweakable Block Cipher with Ciphertext Stealing

2 Functional Overview

This chapter contains a high-level overview of the software and hardware architecture.

2.1 Architecture Overview

EIP-130 is a pipelined design and can have a token in each mailbox while processing one token. The *EIP-130 Driver Library* submits a command token to one of the mailboxes on request from a higher software layer. When the operation is completed, the *EIP-130* signals the Host by updating the status register and asserts a hardware interrupt signal at the same time. The *EIP-130 Driver Library* then reads the result token from the corresponding mailbox.

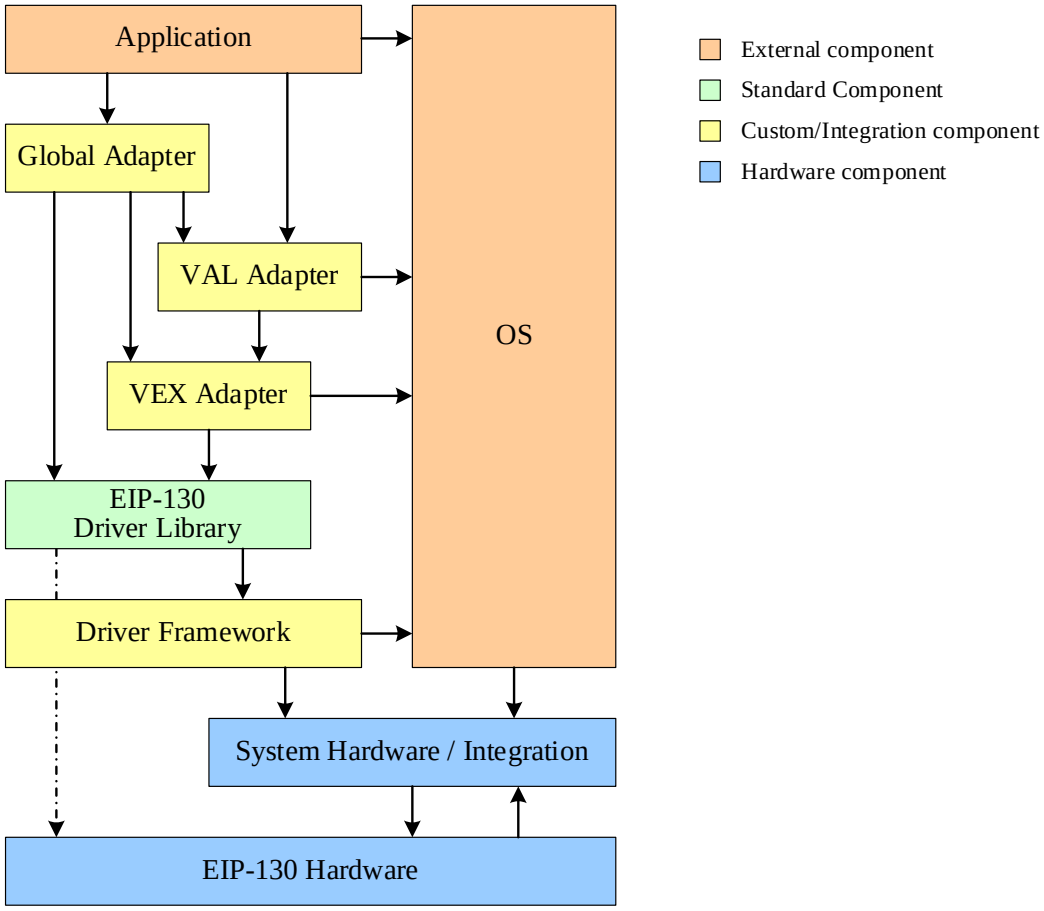


Figure 1 Driver Library Integration Overview

Figure 1 shows the *EIP-130 Driver Library* that is solely responsible for the communications with the *EIP-130* and is used by the 'Adapter' layers. The 'Adapter' layers handle the application requests and perform the following tasks:

- Application interfacing with *EIP-130 Driver Library*
- Concurrent context synchronization
- Dynamic memory allocation

Note: The Adapter layers are not part of the Driver Library and should be custom-made for each environment, together with the Driver Framework implementation. Refer to section 2.3.

The *EIP-130 Driver Library* is a building block that is agnostic to all the typical system issues listed above and abstracts the *EIP-130*, thereby removing the need to understand the *EIP-130* HW registers and commands/responses by higher SW layers.

2.2 EIP-130 HW Structure and Terminology

On top of the *EIP-130* hardware, the *EIP-130 Driver Library* provides an interface to registers intended for general use, Mailboxes intended for Tokens and optionally the Firmware RAM (FWRAM) intended for the General-Purpose Firmware. Figure 2 shows the physical interfaces that the *EIP-130 Driver Library* provides in green and the Token that can be created/parsed with the Token Helper functions provided in the *EIP-130 Driver Library* in blue.

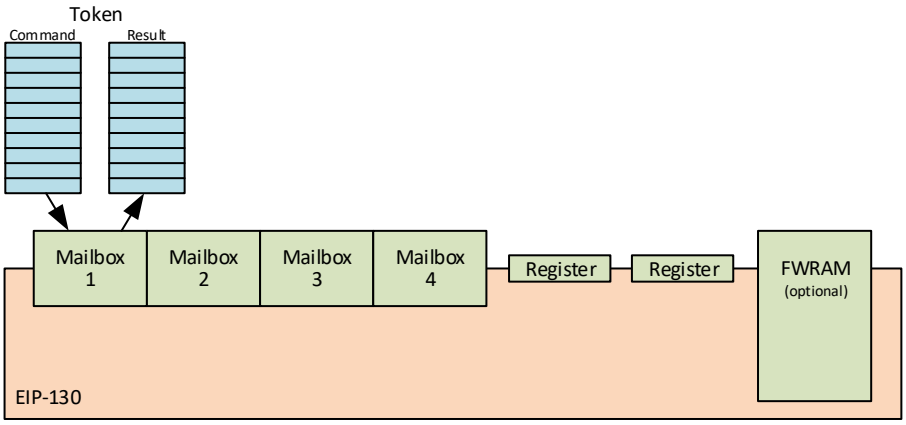


Figure 2 *EIP-130* interface

All *EIP-130* hardware internals are hidden from the outside world. In general, the *EIP-130* hardware reacts on command tokens provided via the Mailboxes and responds to each command token with a result token. This result token holds the actual result and additional information if applicable. When a data buffer (in and/or out) is required in a command, its address is provided in the token and the *EIP-130 Driver Library* has no knowledge of it.

2.3 Driver Framework

The Driver Framework is an important software layer in the driver architecture that makes the Driver Libraries portable across different hardware platforms and OS's. It decouples the *EIP-130 Driver Library* from integration issues (CPU, compiler, Host hardware, OS). This allows the *EIP-130 Driver Library* to be a standard, reusable software component. The Driver Framework is a set of APIs/interfaces. The *EIP-130 Driver Library* makes use of the following four Driver Framework APIs:

- Basic Definitions API: Types with guaranteed storage size and a few support macros
- C Library API: Guarantees the existence of a subset of the C library API
- Device Access API: Access to EIP device registers
- DMA Resource API: Access to DMA Safe Buffers

Driver developers must implement these APIs for their specific Host hardware and OS to allow the *EIP-130 Driver Library* to access the *EIP-130*.

2.4 Driver Library Configuration

The `c_eip130.h` file contains all the configuration parameters for the *EIP-130 Driver Library*. The configuration parameters in this file can be overridden by product-specific parameters specified in the `cs_eip130.h` file. The Driver Library configuration parameters are described in Chapter 4.

2.5 Endianness

All the integers that are written by the *EIP-130 Driver Library* (running on the Host processor) and read by the *EIP-130* or integers that are written by the *EIP-130* and read by the *EIP-130 Driver Library* must be endianness-compatible. There is only one method to ensure the endianness compatibility between the Host processor and the *EIP-130* in case the two devices have different endianness. The method is to let the Host processor swap the bytes within the 32-bit words.

2.5.1 Endianness Conversion by the Host Processor

All I/O operations on static resources (registers) of the *EIP-130* are 32-bit wide and operate through the Driver Framework Device API, of which the implementation handles endianness differences. To access the Driver Framework, the library expects the Adapter to send a valid handle to the `Device_Handle_t` type in a call for the Device API to read and write the *EIP-130* registers, allowing the implementation of the Device API to swap the endianness of the integers, if required.

All I/O operations on dynamic resources (DMA buffers) of *EIP-130* are also 32-bit wide and operate through the Driver Framework, which handles endianness differences. The Driver Library uses the DMAResource API to read and write the *EIP-130* DMA buffers, allowing the implementation of the DMAResource API to swap the endianness of the words, if required.

Refer to the *Driver Framework Porting Guide* [2] for details on how to configure the endianness conversion for the EIP Driver Framework.

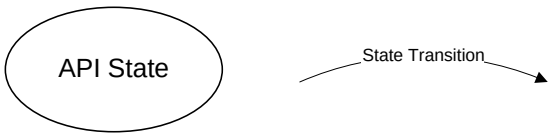
2.6 Concurrent Context Synchronization: Registers and Mailboxes

The *EIP-130* hardware interface is such that it permits very little concurrency for its user on the Host. This means that having multiple execution contexts on the Host all using the *EIP-130* hardware interface will not increase the performance, thus the Driver using the *EIP-130 Driver Library* should serialize the requests. For example, the VEX adapter shown in Figure 1 will normally take care of the request/command serialization. So, from the user's perspective the *EIP-130* interface can be regarded as one execution context.

2.6.1 API Use Order

The *EIP-130 Driver Library* API for Register, Mailbox and Firmware API have a direct involvement with the *EIP-130* hardware, refer to the sections 3.1, 3.2 and 3.3 for the detailed API description. Only the Mailbox API has an API use order that is described in this section. If the VEX adapter is used, top of the *EIP-130 Driver Library* this adapter will take care of the use order.

The state diagrams in this section have the following notations:



The state machine implemented by the *EIP-130 Driver Library* API for the Mailbox API functions is shown in Figure 3. This figure shows the mandatory use order of this API.

Table 1 *EIP-130 Driver Library* Mailbox API State Machine

Function	Allowed in state	Transitions to state
`EIP130_MailboxLink()`	1	2
`EIP130_MailboxUnlink()`	2	1
`EIP130_MailboxCanWriteToken()`	2	Self
`EIP130_MailboxWriteAndSubmitToken()`	2	3
`EIP130_MailboxCanReadToken()`	3	Self
`EIP130_MailboxReadToken()`	3	2
`EIP130_MailboxLinkReset()`[1]	1, 2, 3	1

[1] Only the master Host can call this function without an error.

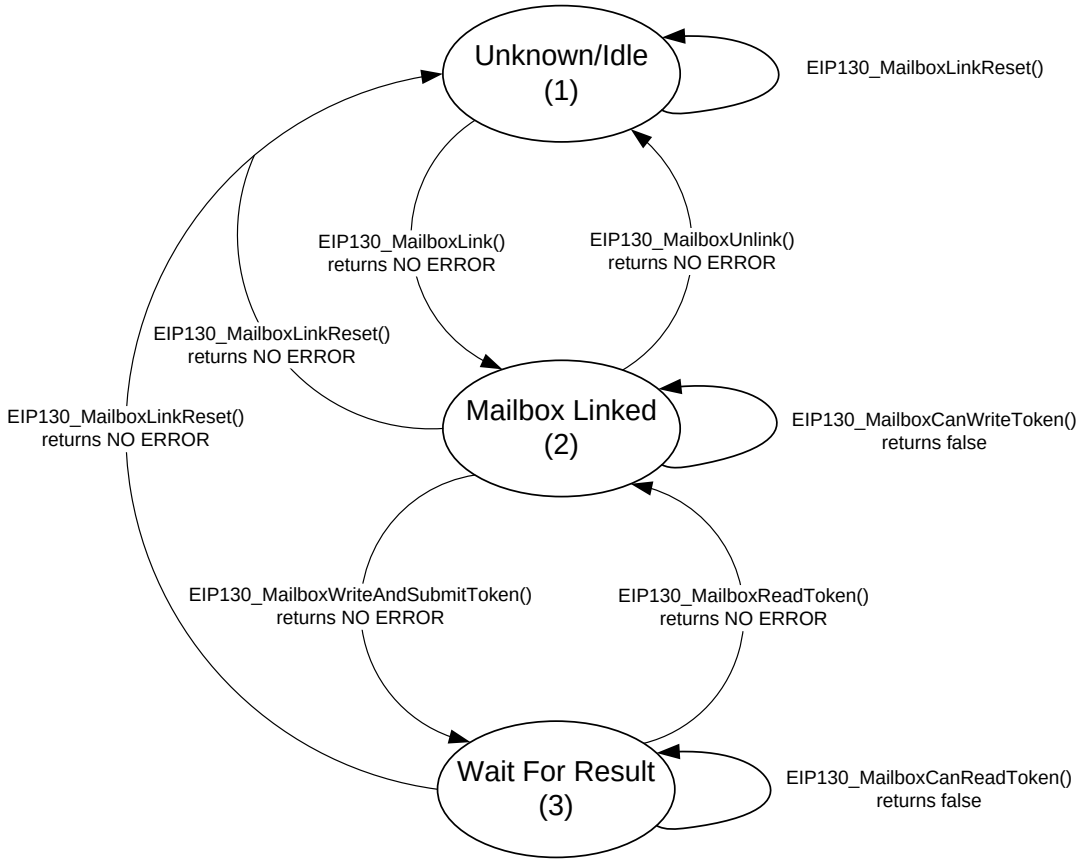


Figure 3 *EIP-130 Driver Library* Mailbox API State Machine

2.7 Concurrent Context Synchronization: Token Helper functions

The Token Helper functions allow concurrency, so they can be called from multiple execution contexts. The Token Helper functions do not call the *EIP-130* hardware directly but they help to assemble an input token or disassemble an output token. Note that an input token is submitted to an *EIP-130* input mailbox and an output token is read from the responding *EIP-130* output mailbox.

2.8 Deployment

The *EIP-130 Driver Library* can be deployed as an integral part of a driver, which can be a kernel-level or user-level module of a general-purpose OS such as Linux or RTOS such as FreeRTOS or VxWorks. The Host HW can be based on a uniprocessor (UP) or multi-processors (AMP, SMP). The Driver Library code and data can be shared by multiple execution contexts. In the case of an AMP, only one Driver Library instance will be running on one processor or one SMP pool of processors making a part of the AMP system. Such a deployment scheme can be performance-sensitive in terms of token processing speed.

The Driver Library can also be used in an OS-less environment, for example in a (Secure) Boot Loader application that require *EIP-130* services. Such a deployment scheme can be footprint-sensitive in terms of binary image size.

The Driver Library code has been optimized for these deployment schemes.

3 Provided Services

This chapter describes the functionality of the API functions and error codes.

3.1 Register API Functions Summary

The Register API provides an abstract reflection of the information that can be written to or read from the device registers. This means for the *EIP-130*:

- reading of the version, options, and status registers.
- reading and writing of the module status register and mailbox related registers that are mainly under control of the Mailbox component.

Based on the above-mentioned information, the Register API will provide the following functions:

Table 2 *EIP-130 Driver Library* Register API

Function	Purpose / Description
`EIP130_RegisterReadVersion()`	Read the hardware version and EIP number.
`EIP130_RegisterReadOptions()`	Read mailbox related information, such as number of mailboxes, mailbox size, master Host ID, calling Host ID, etc.
`EIP130_RegisterReadOptions2()`	Read the module capabilities, such as bus interface type and available engines and configuration.
`EIP130_RegisterReadModuleStatus()`	Read module status, such as the FIPS, CRC24 and firmware image state.
`EIP130_RegisterWriteModuleStatus()`	Write the firmware image written indication.
`EIP130_RegisterReadMailboxStatus()`	Read mailbox status.
`EIP130_RegisterWriteMailboxControl()`	Write mailbox control.
`EIP130_RegisterReadMailboxRawStatus()`	Read raw (unmasked) mailbox status.
`EIP130_RegisterWriteMailboxReset()`	Overriding the 'master' Host mailbox control. ***Note:*** Only the master Host can call this function without an error.
`EIP130_RegisterReadMailboxLinkId()`	Read mailbox linked Host IDs.
`EIP130_RegisterReadMailboxOutId()`	Read mailbox Output Host IDs.
`EIP130_RegisterReadMailboxLockOut()`	Read Host/mailbox lockout control.
`EIP130_RegisterWriteMailboxLockOut()`	Write Host/mailbox lockout control.

For the detailed description of the *EIP-130 Driver Library* Register API refer to [1] and the API is mainly described in the file `eip130_level0.h`.

3.2 Mailbox API Functions Summary

The Mailbox API provides the functionality to write a command token to one of the device mailboxes or to read the corresponding result token. This means for the *EIP-130* in general that the following sequence needs to be executed:

1. Create a command token
2. Link a mailbox
3. Write and submit the command token to the mailbox
4. Wait for the result token
5. Read the result token from the mailbox
6. Unlink the mailbox
7. Process the result token

This sequence is not carved in stone but can be regarded as the one that will work always. Note that the mailbox linking must be successful before a command token can be written and submitted. A sequence in which the mailbox linking is done at driver start and the mailbox unlinking at driver termination will work also.

Table 3 lists the Mailbox API functions. When using these API functions, the above-mentioned sequence and the Concurrent Context Synchronization information mentioned in section 2.6 are important. Additionally, note that the Mailbox API functions will use the Register API as well.

Table 3 *EIP-130 Driver Library* Mailbox API

Function	Purpose / Description
`EIP130_MailboxLink()`	Link specified mailbox.
`EIP130_MailboxUnlink()`	Unlink specified mailbox.
`EIP130_MailboxLinkReset()`	Reset the mailbox link. ***Note:*** Only the master Host can call this function without an error.
`EIP130_MailboxCanWriteToken()`	Check if command token can be written to the mailbox.
`EIP130_MailboxCanReadToken()`	Check if result token can be read from the mailbox.
`EIP130_MailboxWriteAndSubmitToken()`	Write and submit the command to the mailbox.
`EIP130_MailboxReadToken()`	Read the result token from the mailbox.
`EIP130_MailboxAccessVerify()`	Check if the mailbox is accessible.
`EIP130_MailboxAccessControl()`	Control the Host lockout.
`EIP130_MailboxGetLinkID()`	Get information of the Host that linked the mailbox.
`EIP130_MailboxGetOptions()`	Get general mailbox information, such as number of mailboxes, master Host ID and Host ID of the caller.

For the detailed description of the *EIP-130 Driver Library* Mailbox API refer to [1] and the API is mainly described in the file `eip130.h`.

3.3 Firmware API Functions Summary

The Firmware API provides the functionality to check, load and boot the *EIP-130* General-Purpose Firmware image if *EIP-130* hardware configuration requires it.

Note: The firmware load procedure depends on the EIP-130 hardware used, so make sure that the correct hardware version is selected in the `cs_eip130.h` file, please refer to section 4.

Table 4 lists the Firmware API functions. Note that the Firmware API functions will use the Register and Mailbox API as well.

Table 4 *EIP-130 Driver Library* Firmware API

Function	Purpose / Description
`EIP130_FirmwareCheck()`	Checks if *EIP-130* has a usable firmware image.
`EIP130_FirmwareLoad()`	Loads and boots the firmware image.
`EIP130_ModuleGetOptions()`	Retrieves specific module hardware related information.
`EIP130_ModuleGetStatus()`	Retrieves the module status information.
`EIP130_ModuleFirmwareWritten()`	Writes the indication that the firmware is written in FWRAM.

For the detailed description of the *EIP-130 Driver Library* Firmware API refer to [1] and the API is mainly described in the file `eip130.h`.

3.4 Token Helper API Summary

The Token Helper API consists of functions and Asset Policy defines that can be used in upper-level code. For the detailed description of the *EIP-130 Driver Library* Token Helper API refer to [1].

Table 5 lists all *EIP-130 Driver Library* Token Helper functions that are roughly grouped based on their functionality and in functions that work on command tokens and functions that work on result tokens. These functions will be expanded inline in the code except for the functions `Eip130Token_Command_WriteByteArray()` and `Eip130Token_Result_ReadByteArray()`, which are functions of their own. The functions are described in the files `eip130_token_*.h` in more detail.

The Asset Policy defines are not listed but can be found in the file `eip130_asset_policy.h`. Note that the applicable defines depend on the *EIP-130* FW used.

Table 5 *EIP-130 Driver Library* Token Helper API

Function	Purpose / Description
General functions	
`Eip130Token_Command_Clear()`	Clear the command token buffer ***Note:*** This function does not zeroize all command words but initializes them with a known pattern (0xAAAAAAAA).
`Eip130Token_Command_Identity()`	Set the Identity in the command token.
`Eip130Token_Command_SetTokenId()`	Set the Token Identifier in the command token.
`Eip130Token_Command_WriteByteArray()`	Write an array of bytes to certain command token location.
`Eip130Token_Result_Code()`	Retrieves result information from the result token and returns it via the function return: 0 Successful operation (no error or warning) <0 Error >0 Warning
`Eip130Token_Result_ReadByteArray()`	Read an array of bytes from a certain result token location.
NOP	
`Eip130Token_Command_NOP()`	Initialize command token for a NOP operation.
Encrypt	
`Eip130Token_Command_Crypto_Operation()`	Initialize command token for the decrypt or encrypt operation of a specific algorithm (AES, DES, 3DES, ChaCha20, SM4 and ARIA).
`Eip130Token_Command_Crypto_SetData Addresses()`	Set the Data address for Input and Output.
`Eip130Token_Command_Crypto_SetADAddress()`	Sets the Associated Data address and length in the command token.
`Eip130Token_Command_Crypto_SetKeyLength()`	Set the key length in the command token for an AES or ARIA operation.
`Eip130Token_Command_Crypto_ChaCha20_Set KeyLength()`	Set the key length in the command token for a ChaCha20 operation.
`Eip130Token_Command_Crypto_SetNonceLength()`	Set the nonce length in the command token for a ChaCha20 operation.
`Eip130Token_Command_Crypto_SetGcmMode()`	Sets the GCM operation mode in the command token. ***Note:*** Only applicable for EIP-130 HW2.x.
`Eip130Token_Command_Crypto_SetNoRandomIV()`	Sets the NoRandomIV indication for a GCM operation in the command token to indicate the IV from token shall be used.
`Eip130Token_Command_Crypto_SetASLoadKey()`	Set Asset Store Identifier of key in the command token. ***Note:*** Should not be used.in combination with `Eip130Token_Command_Crypto_CopyKey()`.

Function	Purpose / Description
Eip130Token_Command_Crypto_SetASLoadIV()	Set Asset Store Identifier of the intermediate IV/Counter in the command token. ***Note:*** Should not be used.in combination with Eip130Token_Command_Crypto_CopyIV().
Eip130Token_Command_Crypto_SetASSaveIV()	Set Asset Store Identifier of the intermediate/final IV/counter in the command token.
Eip130Token_Command_Crypto_CopyKey()	Copy the key to the command token. ***Note:*** Should not be used.in combination with Eip130Token_Command_Crypto_SetASLoadKey().
Eip130Token_Command_Crypto_CopyIV()	Copy the previous intermediate IV/counter to the command token. ***Note:*** Should not be used.in combination with Eip130Token_Command_Crypto_SetASLoadIV().
Eip130Token_Command_Crypto_CopyNonce()	Copy the Nonce to the command token.
Eip130Token_Command_Crypto_CopyHashKey()	Copy the Hash key to the command token. ***Note:*** Only applicable for EIP-130 HW2.x.
Eip130Token_Command_Crypto_CopyTag()	Copy the TAG to the command token.
Eip130Token_Command_Crypto_Copyf8Salt Key()	Copy the AES-f8 salt key to the command token.
Eip130Token_Command_Crypto_Copyf8IV()	Copy the AES-f8 IV to the command token.
Eip130Token_Command_Crypto_CopyParameter ()	Copy the (SM4) parameter to the command token.
Eip130Token_Result_Encrypt_CopyIV()	Copy the intermediate/final IV/counter from the result token.
Eip130Token_Result_Encrypt_CopyTag()	Copy the TAG from the result token.
Hash	
Eip130Token_Command_Hash()	Initialize command token for a Hash operation
Eip130Token_Command_Hash_SetStateASID()	Set intermediate state Asset Store Identifier in the command token.
Eip130Token_Command_Hash_SetTotalMessage Length()	Set the total message length of the hash operation in the command token. ***Note:*** Eip130Token_Command_Hash() will not set this length.
Eip130Token_Command_Hash_CopyState()	Copy the previous intermediate state to the command token.
Eip130Token_Result_Hash_CopyState()	Copy the intermediate state/final digest from the result token.
MAC	
Eip130Token_Command_MAC()	Initialize command token for a MAC operation.
Eip130Token_Command_Mac_SetASTempMAC()	Set intermediate state Asset Store Identifier in the command token. ***Note:*** Only applicable when the EIP-130 HW does not support SHA-3.
Eip130Token_Command_Mac_SetASIDState()	Set intermediate state Asset Store Identifier in the command token. ***Note:*** Only applicable when the EIP-130 HW supports SHA-3.
Eip130Token_Command_Mac_SetASLoadKey()	Set Asset Store Identifier of key in the command token. ***Note:*** Only applicable when the EIP-130 HW does not support SHA-3 and should not be used.in combination with Eip130Token_Command_Mac_CopyKey().
Eip130Token_Command_Mac_SetASIDKey()	Set Asset Store Identifier of key in the command token. ***Note:*** Only applicable when the EIP-130 HW supports SHA-3.

Function	Purpose / Description
Eip130Token_Command_Mac_SetASLoadMAC()	Set Asset Store Identifier of the final MAC to verify in the command token. ***Note:*** Only applicable when the EIP-130 HW does not support SHA-3 and should not be used.in combination with Eip130Token_Command_Mac_CopyMAC().
Eip130Token_Command_Mac_SetASIDVerify MAC()	Set Asset Store Identifier of the final MAC to verify in the command token. ***Note:*** Only applicable when the EIP-130 HW supports SHA-3 and should not be used.in combination with Eip130Token_Command_Mac_CopyVerifyMAC().
Eip130Token_Command_MAC_SetTotalMessage Length()	Set the total message length of the MAC operation in the command token. ***Note:*** Eip130Token_Command_MAC() will not set this length.
Eip130Token_Command_Mac_CopyKey()	Copy the key to the command token. ***Note:*** Only applicable when the EIP-130 HW does not support SHA-3 and should not be used.in combination with Eip130Token_Command_Mac_SetASLoadKey().
Eip130Token_Command_Mac_CopyMAC()	Copy the previous intermediate state or MAC to verify to the command token. ***Note:*** Only applicable when the EIP-130 HW does not support SHA-3 and should not be used.in combination with Eip130Token_Command_Mac_SetASTempMAC() or Eip130Token_Command_Mac_SetASLoadMAC().
Eip130Token_Command_Mac_CopyVerifyMAC()	Copy the MAC to verify to the command token. ***Note:*** Only applicable when the EIP-130 HW supports SHA-3 and should not be used.in combination with Eip130Token_Command_Mac_SetASIDVerifyMAC().
Eip130Token_Result_Mac_CopyMAC()	Copy the intermediate state/final MAC from the result token. ***Note:*** Only applicable when the EIP-130 HW does not support SHA-3.
Eip130Token_Result_Mac_CopyFinalMAC()	Copy the final MAC from the result token. ***Note:*** Only applicable when the EIP-130 HW supports SHA-3.
TRNG	
Eip130Token_Command_TRNG_Configure()	Initialize command token for a TRNG configuration operation, which default set for AES-CTR-DRBG. ***Note:*** Only the Crypto Officer can do this operation.
Eip130Token_Command_TRNG_Configure_Aria DRBG()	Set the ARIA-CTR-DRBG indication in the command token for the TRNG configuration operation. ***Note:*** Only the Crypto Officer can do this operation and the EIP-130 HW shall support ARIA.
Eip130Token_Command_DRBG_ForceSeed()	Initialize command token for a TRNG configuration operation for DRBG force seed. ***Note:*** When the FW provides this functionality, only the Crypto Officer can do this operation and it disables the TRNG use.
Eip130Token_Command_PRNG_ReseedNow()	Initialize command token for a PRNG Reseed operation.
Eip130Token_Command_RandomNumber_Generate ()	Initialize command token for a Random Number generation operation.
Eip130Token_Command_RandomNumber_SetRaw Key()	Set the RawKey value in the command token for a Random Number generation operation to indicate that raw data shall be fetched.

Function	Purpose / Description
Eip130Token_Result_RandomNumber_Generate()	Extracts 'quality warning' details from the result token. ***Note:*** Eip130Token_Result_Result() could be used also.
Eip130Token_Command_TRNG_PP_Verification()	Initialize command token for the TRNG post-processing verification.
Eip130Token_Command_TRNG_HW_SelfTest()	Initialize command token for the TRNG hardware self-test verification.
Eip130Token_Result_TRNG_HW_SelfTest()	Extracts the TRNG hardware self-test verification information from the result token.
Symmetric Key Wrap	
Eip130Token_Command_WrapUnwrap()	Initialize command token for a key wrap or unwrap operation.
Eip130Token_Command_WrapUnwrap_SetAlgorithm()	Set the algorithm in the command token. (AES or SM4)
Eip130Token_Command_WrapUnwrap_SetKeyLength()	Set the key length in the command token.
Eip130Token_Command_WrapUnwrap_SetASLoadKey()	Set Asset Store Identifier of key in the command token. ***Note:*** Should not be used.in combination with Eip130Token_Command_WrapUnwrap_CopyKey().
Eip130Token_Command_WrapUnwrap_CopyKey()	Copy the key to the command token. ***Note:*** Should not be used.in combination with Eip130Token_Command_WrapUnwrap_SetLoadKey().
Eip130Token_Result_WrapUnwrap_GetDataLength()	Read the length of the wrapped or unwrapped data from the result token.
Eip130Token_Command_EncryptedVectorPKI()	Initialize command token for an encrypted vector for PKI operation. ***Note:*** Only applicable when the EIP-130 FW supports this.
Eip130Token_Result_EncryptedVectorPKI_GetDataLength()	Read the length of the encrypted vector data from the result token. ***Note:*** Only applicable when the EIP-130 FW supports this.
Special Functions for Milenage	
Note: Only applicable when the EIP-130 FW supports Milenage.	
Eip130Token_Command_SF_MilenageSqnAdminCreate()	Initialize command token for creating a SQN_{MS} Administration Asset.
Eip130Token_Result_SF_MilenageSqnAdminCreate()	Reads the SQN_{MS} Administration Asset Store Identifier from the result token.
Eip130Token_Command_SF_MilenageSqnAdminReset()	Initialize command token for resetting a SQN_{MS} Administration Asset to initial state.
Eip130Token_Command_SF_MilenageSqnAdminExport()	Initialize command token for exporting a SQN_{MS} Administration Asset.
Eip130Token_Result_SF_MilenageSqnAdminExport()	Reads the length of the data blob from the result token.
Eip130Token_Command_SF_MilenageAutnVerification()	Initialize command token for AUTN verification without SQN_{MS} Administration use.
Eip130Token_Command_SF_MilenageAutnVerificationSqn()	Initialize command token for AUTN verification with SQN_{MS} Administration use.
Eip130Token_Result_SF_MilenageAutnVerification()	Reads the RES, CK and IK from the result token.
Eip130Token_Result_SF_MilenageAutnVerificationEMMCause()	Reads the EMM Cause from the result token.
Eip130Token_Result_SF_MilenageAutnVerificationSQNAMF()	Reads the SQN and AMF from the result token. ***Note:*** Only applicable for AUTN verification without SQN_{MS} Administration use.

Function	Purpose / Description
Eip130Token_Command_SF_MilenageAuts Generation()	Initialize command token for AUTS generation.
Eip130Token_Result_SF_MilenageAuts()	Reads the generated AUTS from the result token.
Eip130Token_Command_SF_Milenage Conformance()	Initialize command token for a Milenage Conformance test.
Eip130Token_Result_SF_Milenage Conformance()	Reads the Conformance result data from the result token.
Asset Management	
Eip130Token_Command_AssetSearch()	Initialize command token for a static Asset search operation.
Eip130Token_Result_AssetSearch()	Retrieve the Asset Store Identifier and Asset length from the result token.
Eip130Token_Command_AssetCreate()	Initialize command token for a dynamic Asset creation operation.
Eip130Token_Command_AssetCreate_SetAll Hosts()	Sets the AllHosts indication in the command token. ***Note:*** Only applicable for EIP-130 FW2.x.
Eip130Token_Command_AssetCreate_SetRemove Secure()	Sets the RemoveSecure indication in the command token.
Eip130Token_Command_AssetCreate_Set Lifetime()	Sets the lifetime information in the command token. ***Note:*** Only applicable when the EIP-130 FW supports this.
Eip130Token_Result_AssetCreate()	Retrieve the Asset Store Identifier from the result token.
Eip130Token_Command_AssetDelete()	Initialize command token for a dynamic Asset deletion operation.
Eip130Token_Command_AssetLoad_Derive()	Initialize command token for a dynamic Asset load operation where the Asset is initialized with data that derived from the specified key.
Eip130Token_Command_AssetLoad_Random()	Initialize command token for a dynamic Asset load operation where the Asset is initialized with random data.
Eip130Token_Command_AssetLoad_Plaintext()	Initialize command token for a dynamic Asset load operation where the Asset is initialized with data that is provided in plaintext.
Eip130Token_Command_AssetLoad_Import()	Initialize command token for a dynamic Asset load operation where the Asset is initialized with data from a key blob that was generated previously.
Eip130Token_Command_AssetLoad_SymUnwrap()	Initialize command token for a dynamic Asset load operation where the Asset is initialized with data from a symmetric wrapped key blob with AES or SM4 Algorithm.
Eip130Token_Command_AssetLoad_SetAad()	Copy the Associate Data to the command token.
Eip130Token_Command_AssetLoad_SetInput()	Set the input data related information in the command token.
Eip130Token_Command_AssetLoad_SetOutput()	Set the output data related information in the command token.
Eip130Token_Command_AssetLoad_Export()	Set the export related information in the command token.
Eip130Token_Result_AssetLoad_OutputSize()	Retrieve the output data (key blob) length from the result token.
Eip130Token_Command_SecureTimer()	Initialize command token for Secure Timer (Re)Start, Stop and read operation.
Eip130Token_Result_SecureTimer()	Retrieve the Asset Store Identifier of the Secure Timer and elapsed time from the result token.
Eip130Token_Command_PublicData_Read()	Initialize command token for a public data object read operation.
Eip130Token_Result_Publicdata_Read()	Retrieve the data length of the Public Data object from the result token.
Eip130Token_Command_OTPMonotonicCounter_ Read()	Initialize command token for Monotonic Counter read operation.

Function	Purpose / Description
Eip130Token_Result_OTPMonotonicCounter_Read()	Retrieve the data length of the Monotonic Counter from the result token.
Eip130Token_Command_OTPMonotonicCounter_Increment()	Initialize command token for Monotonic Counter increment operation. ***Note:*** Everyone can increment a monotonic counter with one exception, the 32-bit one (Asset number 32) can only be incremented by the Crypto Officer.
Eip130Token_Command_OTPDataWrite()	Initialize command token for One-Time-Programmable write operation. ***Note:*** Only the Crypto Officer can do this operation.
Eip130Token_Command_ProvisionRandomHUK()	Initialize command token for the operation that provisions a randomly generated HUK (Root Key) and the specified COID in OTP. ***Note:*** The operation is only successful if the OTP is empty.
Eip130Token_Result_ProvisionRandomHUK()	Retrieve the OTP key blob length of the HUK from the result token.
Eip130Token_Command_AssetExport_Coprocessor()	Exports the specified Asset via the Co-Processor interface. ***Note:*** Only applicable when the EIP-130 HW/FW supports this.
Eip130Token_Command_AssetStore_Reset()	Reset/Clear the dynamic Asset Store. ***Note:*** Only applicable when the EIP-130 FW supports this.
Authentication Unlock	
Eip130Token_Command_AUnlock_Start()	Initialize command token for the start of Authentication Unlock operation.
Eip130Token_Result_AUnlock_CopyNonce()	Retrieve the Nonce from the result token.
Eip130Token_Command_AUnlock_Verify()	Initialize command token for the verification of Authentication Unlock operation.
Eip130Token_Command_SetSecureDebug()	Initialize command token for a Secure Debug operation to enable or disable debug functionality.
Public Key	
Eip130Token_Command_Pk_Asset_Command()	Initialize command token for the start of Public Key operation with Asset Store use.
Eip130Token_Command_Pk_Asset_SetAdditionalData()	Copies the additional input data to the command token.
Eip130Token_Command_Pk_Asset_SetAdditionalOutputData()	Copies additional output data to the command token.
Eip130Token_Command_Pk_Asset_SetAdditionalAssetId()	Copies the specified AssetId to the additional data area of the command token.
Eip130Token_Command_Pk_Asset_SetAdditionalLength()	Copies the specified length to the additional data area of the command token.
Eip130Token_Command_Pk_Asset_AddlenCorrection()	Corrects the AddLen in the command token.
Eip130Token_Command_Pk_Asset_SaveSharedSecret()	Set the Save Shared Secret indication in the command token.
Eip130Token_Command_Pk_Asset_SetExplicitDigest()	Set the Explicit Digest indication in the command token.
Eip130Token_Command_Pk_Claim()	Initialize command token for the start of Public Key operation without Asset Store use, which claims the PKA engine.
Eip130Token_Command_Pk_NumLoad()	Initialize command token for the vector load of Public Key operation without Asset Store use.
Eip130Token_Command_Pk_Operation()	Initialize command token for the function execution of Public Key operation without Asset Store use.

Function	Purpose / Description
EIP130Token_Command_PK_Release()	Initialize command token for the release of Public Key operation without Asset Store use, which releases the PKA engine.
eMMC	
Note: Only applicable when the EIP-130 FW supports this.	
Eip130Token_Command_eMMC_ReadRequest()	Initialize command token for the eMMC Read Request operation.
Eip130Token_Command_eMMC_ReadWriteCounterRequest()	Initialize command token for the eMMC Read Write-Counter Request operation.
Eip130Token_Result_eMMC_ReadStateID	Retrieve the eMMC State Asset ID from the result token.
Eip130Token_Result_eMMC_ReadCopyNonce()	Copy the Nonce from the result token.
Eip130Token_Command_eMMC_ReadVerify()	Initialize command token for the eMMC Read Verify operation.
Eip130Token_Command_eMMC_ReadWriteCounterVerify()	Initialize command token for the eMMC Read Write-Counter Verify operation.
Eip130Token_Command_eMMC_WriteRequest()	Initialize command token for the eMMC Write Request operation.
Eip130Token_Command_eMMC_WriteVerify()	Initialize command token for the eMMC Write Verify operation.
Eip130Token_Command_eMMC_SetStateAssetID()	Set the eMMC State Asset ID in the command token.
Eip130Token_Command_eMMC_SetInputDataAndMACInfo()	Set the input data and MAC related information in the command token.
Eip130Token_Command_eMMC_CopyMAC()	Copy the MAC value to the command token.
Eip130Token_Result_eMMC_CopyMAC()	Retrieve the MAC value from the result token.
Service	
Eip130Token_Command_RegisterRead()	Initialize command token for the register read operation.
Eip130Token_Result_RegisterRead()	Retrieve the read value from the result token.
Eip130Token_Command_RegisterWrite()	Initialize command token for the register write operation. ***Note:*** Only the Crypto Officer can do this operation.
Eip130Token_Command_ZeroizeOutputMailbox()	Initialize command token for the zeroize output mailbox operation.
Eip130Token_Command_ClockSwitch()	Initialize command token for the Clock Switch operation. ***Note:*** Only the Crypto Officer can do this operation.
Eip130Token_Command_OTPSelectZeroize()	Initialize command token for the operation to select the OTP Zeroize. ***Note:*** Only the Crypto Officer can do this operation.
Eip130Token_Command_OTPZeroize()	Initialize command token for the actual OTP Zeroize operation. Note that the operation must be preceded by the OTP Select Zeroize operation. ***Note:*** Only the Crypto Officer can do this operation.
Eip130Token_Command_FirmwareCheck()	Initialize command token for verifying a firmware image. ***Note:*** Only applicable when using FWRAM.
Eip130Token_Command_UpdateRollbackID()	Initialize command token with a new value for the SFWVBVER monotonic counter. ***Note:*** Only applicable when using FWRAM.
System	
Eip130Token_Command_SystemInfo()	Initialize command token for the system information operation.
Eip130Token_Result_SystemInfo()	Retrieve the system information from the result token.

Function	Purpose / Description
`Eip130Token_Command_SystemSelfTest()`	Initialize command token for the Self-Test operation. ***Note:*** For EIP-130 HW2.x, this operation also performs the transition to FIPS mode.
`Eip130Token_Command_EnterFipsMode()`	Initialize command token for the enter-FIPS-Mode operation. ***Note:*** Only applicable for EIP-130 HW3.x.
`Eip130Token_Command_SystemReset()`	Initialize command token for the Reset operation.
`Eip130Token_Command_SystemDefineUsers()`	Initialize command token for the operation to define the FIPS Users. ***Note:*** Only the Crypto Officer can do this operation.
`Eip130Token_Command_SystemSetTime()`	Initialize command token for the operation to set the system time. ***Note:*** Only the Crypto Officer can do this operation, if the EIP-130 FW supports this.
`Eip130Token_Command_SystemSleep()`	Initialize command token for the Sleep operation.
`Eip130Token_Command_SystemResumeFrom Sleep()`	Initialize command token for the Resume-From-Sleep operation.
`Eip130Token_Command_SystemHibernation()`	Initialize command token for the Hibernation operation. ***Note:*** Only applicable when the EIP-130 FW supports this.
`Eip130Token_Command_SystemResumeFrom Hibernation()`	Initialize command token for the Resume-From-Hibernation operation. ***Note:*** Only applicable when the EIP-130 FW supports this.
`Eip130Token_Command_SystemHibernation Infomation()`	Initialize the general part of the Hibernation or Resume-From-Hibernation command token. ***Note:*** Only applicable when the EIP-130 FW supports this.
`Eip130Token_Result_SystemHibernation_Blob Size()`	Retrieve the size of data blob from the result token. ***Note:*** Only applicable when the EIP-130 FW supports this.

4 Configuration Parameters

This chapter describes all *EIP-130 Driver Library* configuration parameters in detail. The default parameters are set in the `c_eip130.h` file and can be re-defined in the top-level configuration file `cs_eip130.h`. The following parameters can be set in this configuration file. For the complete list of available parameters, refer to these configuration files.

Attention: Please contact Rambus technical support for the `cs_eip130.h` file that matches your *EIP-130* HW and FW configuration because the default provided one will probably not match.

4.1 General settings

4.1.1 EIP130_STRICT_ARGS

When this define is set, strict argument checking is enabled in all Register, Mailbox and Firmware API functions of the *EIP-130 Driver Library*.

4.1.2 EIP130_HW_V2

When this define is set, the *EIP-130 Driver Library* APIs are setup to meet the hardware version 2 requirements/behavior.

4.1.3 EIP130_HW_V3

When this define is set, the *EIP-130 Driver Library* APIs are setup to meet the hardware version 3 requirements/behavior.

4.1.4 EIP130_ENABLE_FIRMWARE_LOAD

When this define is set, it indicates that the *EIP-130* HW requires General-Purpose firmware download functionality.

Note: This define is not directly used in the *EIP-130 Driver Library*, but is used by the upper-level code.

4.1.5 EIP130_ALLOW_MASTER_NONSECURE

When this define is set, the *EIP-130 Driver Library* is setup to allow a non-secure master Host.

4.2 Token Helper functions

4.2.1 NOP: EIP130_ENABLE_NOP

When this define is set, the NOP functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.2 Encryption: EIP130_ENABLE_SYM_ALGO_AES

When this define is set, the AES related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.3 Encryption: EIP130_ENABLE_SYM_ALGO_AES_XTS

When this define is set, the AES-XTS related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.4 Encryption: EIP130_ENABLE_SYM_ALGO_AES_CCM

When this define is set, the AES-CCM related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.5 Encryption: EIP130_ENABLE_SYM_ALGO_AES_GCM

When this define is set, the AES-GCM related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.6 Encryption: EIP130_ENABLE_SYM_ALGO_AES_F8

When this define is set, the AES-*f*8 related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.7 Encryption: EIP130_ENABLE_SYM_ALGO_DES

When this define is set, the DES related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.8 Encryption: EIP130_ENABLE_SYM_ALGO_3DES

When this define is set, the Triple-DES/3DES related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.9 Encryption: EIP130_ENABLE_SYM_ALGO_CHACHA20

When this define is set, the ChaCha20 related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.10 Encryption: EIP130_ENABLE_SYM_ALGO_ARIA

When this define is set, the ARIA related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.11 Encryption: EIP130_ENABLE_SYM_ALGO_ARIA_CCM

When this define is set, the ARIA-CCM related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.12 Encryption: EIP130_ENABLE_SYM_ALGO_ARIA_GCM

When this define is set, the ARIA-GCM related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.13 Encryption: EIP130_ENABLE_SYM_ALGO_SM4

When this define is set, the SM4 related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.14 Hash/MAC: EIP130_ENABLE_HASHMAC_LAYOUT_V2

When this define is set, the *EIP-130 Driver Library* generates hash and MAC tokens according to token layout V2.

4.2.15 Hash: EIP130_ENABLE_SYM_ALGO_SHA512

When this define is set, the SHA-512 related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.16 Hash: EIP130_ENABLE_SYM_ALGO_SHA3

When this define is set, the SHA-3 related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.17 Hash: EIP130_ENABLE_SYM_ALGO_SM3

When this define is set, the SM3 related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.18 MAC: EIP130_ENABLE_SYM_ALGO_POLY1305

When this define is set, the Poly1305 related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.19 TRNG: EIP130_ENABLE_TRNG_CUTOFF

When this define is set, the *EIP-130 Driver Library* selects the token helper functions that have cutoff values arguments.

4.2.20 TRNG: EIP130_ENABLE_DRBG_FORCESEED

When this define is set, the *EIP-130* FW supports the functionality to force seed the DBRG, which disables the use of the TRNG (NRBG). The Token Helper API related function is enabled in the *EIP-130 Driver Library* accordingly.

4.2.21 TRNG: EIP130_ENABLE_SYM_ALGO_ARIA_DRBG

When this define is set, the ARIA-DRBG related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.22 Special Functions: EIP130_ENABLE_SF_MILENAGE

When this define is set, the Special Function for Milenage related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.23 Symmetric Key Wrap: EIP130_ENABLE_SYM_KEYWRAP

When this define is set, the Symmetric-key wrap related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.24 Symmetric Key Wrap: EIP130_ENABLE_ENCRYPTED_VECTOR

When this define is set, the encrypted vector for PKI related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.25 Asset Management: EIP130_FW_ASSET_LIFETIME_MANAGEMENT

When this define is set, it indicates that the Asset Store supports Asset lifetime functionality is enabled.

4.2.26 Asset Management: EIP130_FW_ASSETPOLICY_V2

When this define is set, the *EIP-130* FW supports Asset Policy version 2, which is the newest one and requires *EIP-130* HW version 3. The Token Helper API related functions and defines are enabled in the *EIP-130 Driver Library* accordingly.

4.2.27 Asset Management: EIP130_ENABLE_CPIF

When this define is set, the Asset export via the CoProcessor Interface related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.28 Asset Management: EIP130_ENABLE_ASSET_STORE_RESET

When this define is set, the Asset Store reset related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.29 Public Key: EIP130_ENABLE_EXPLICITDIGEST

When this define is set, the explicit digest related functionality for sign/verify operations is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.30 Public Key: EIP130_ENABLE_ASYM_ALGO_DH

When this define is set, the DH (Diffie-Hellman) related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.31 Public Key: EIP130_ENABLE_SYM_ALGO_ECDH

When this define is set, the ECDH related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.32 Public Key: EIP130_ENABLE_ASYM_ALGO_CURVE25519

When this define is set, the Curve25519 related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.33 Public Key: EIP130_ENABLE_ASYM_ALGO_ECDSA

When this define is set, the ECDSA related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.34 Public Key: EIP130_ENABLE_ASYM_ALGO_EDDSA

When this define is set, the EdwardsDSA related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.35 Public Key: EIP130_ENABLE_ASYM_ALGO_ECC_ELGAMAL

When this define is set, the ECC El-Gamal related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.36 Public Key: EIP130_ENABLE_ASYM_ALGO_RSA

When this define is set, the RSA related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.37 Public Key: EIP130_ENABLE_ASYM_ALGO_ECIES

When this define is set, the ECIES functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.38 Public Key: EIP130_ENABLE_SYM_ALGO_SM2

When this define is set, the SM2 related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.39 eMMC: EIP130_ENABLE_EMMC

When this define is set, the eMMC related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.40 External Service: EIP130_ENABLE_EXTSERVICE

When this define is set, the external service-related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

Note: External service is based on functionality provided by eMMC.

4.2.41 System: EIP130_ENABLE_FIRMWARE_SLEEP

When this define is set, the *EIP-130* sleep related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

4.2.42 System: EIP130_ENABLE_FIRMWARE_HIBERNATION

When this define is set, the *EIP-130* hibernation related functionality is enabled in the functions of the Token Helper API of the *EIP-130 Driver Library*.

Note: Hibernation is based on functionality provided by eMMC.

4.2.43 System: EIP130_ENABLE_SYSTEM_ENTERFIPSMODE

When this define is set, the *EIP-130* FW supports the Enter-FIPS-Mode token, which performs the transition to FIPS mode instead of the Self-Test token. The Token Helper API related function is enabled in the *EIP-130 Driver Library* accordingly.

4.3 Register, Mailbox and Firmware API functions

4.3.1 EIP130_REMOVE_MAILBOXACCESSVERIFY

When this define is set, the function for verifying the mailbox access is removed as API function of the Mailbox API of the *EIP-130 Driver Library*.

4.3.2 EIP130_REMOVE_MAILBOXGETOPTIONS

When this define is set, the function for reading the mailbox options is removed as API function of the Mailbox API of the *EIP-130 Driver Library*.

4.3.3 EIP130_REMOVE_MAILBOXACCESSCONTROL

When this define is set, the functions for to manipulate the mailbox access are removed as API functions of the Mailbox API of the *EIP-130 Driver Library*.

4.3.4 EIP130_REMOVE_MAILBOXLINK

When this define is set, the function for linking the mailbox is removed as API function of the Mailbox API of the *EIP-130 Driver Library*.

4.3.5 EIP130_REMOVE_MAILBOXLINKRESET

When this define is set, the function for resetting the mailbox link state is removed as API function of the Mailbox API of the *EIP-130 Driver Library*.

4.3.6 EIP130_REMOVE_MAILBOXUNLINK

When this define is set, the function for unlinking the mailbox is removed as API function of the Mailbox API of the *EIP-130 Driver Library*.

4.3.7 EIP130_REMOVE_MAILBOXCANWRITETOKEN

When this define is set, the function for checking if a token can be written to the mailbox is removed as API function of the Mailbox API of the *EIP-130 Driver Library*.

4.3.8 EIP130_REMOVE_MAILBOXRAWSTATUS

When this define is set, the function for reading the raw mailbox status is removed as API function of the Mailbox API of the *EIP-130 Driver Library*.

4.3.9 EIP130_REMOVE_MAILBOXRESET

When this define is set, the function for resetting the mailbox is removed as API function of the Mailbox API of the *EIP-130 Driver Library*.

4.3.10 EIP130_REMOVE_MAILBOXLINKID

When this define is set, the function for reading the link ID is removed as API function of the *EIP-130 Driver Library*.

4.3.11 EIP130_REMOVE_MAILBOXOUTID

When this define is set, the function for reading the out ID is removed as API function of the *EIP-130 Driver Library*.

4.3.12 EIP130_REMOVE_FIRMWARECHECK

When this define is set, the functions related to General-Purpose firmware check are removed as API functions of the *EIP-130 Driver Library*.

4.3.13 EIP130_REMOVE_FIRMWAREDOWNLOAD

When this define is set, the functions related to General-Purpose firmware image load and start are removed as API functions of the *EIP-130 Driver Library*.

4.3.14 EIP130_REMOVE_MODULEFIRMWAREWRITTEN

When this define is set, the functions related to write the General-Purpose firmware written indication are removed as API functions of the *EIP-130 Driver Library*.

4.3.15 EIP130_REMOVE_MODULEGETOPTIONS

When this define is set, the function to read the module options register is removed as API function of the *EIP-130 Driver Library*.

4.3.16 EIP130_REMOVE_MODULEGETSTATUS

When this define is set, the function to read the module status register is removed as API function of the *EIP-130 Driver Library*.

4.3.17 EIP130_REMOVE_VERSION_OPTION

When this define is set, the function to read the *EIP-130* version is removed as API function of the *EIP-130 Driver Library*.

(End of Document)